Use Cases for Thread-Local Storage

ISO/IEC JTC1 SC22 WG21 P0097R0 - 2015-09-24

Paul E. McKenney, paulmck@linux.vnet.ibm.com
JF Bastien, jfb@google.com
Pablo Halpern, phalpern@halpernwightsoftware.com
Michael Wong, michaelw@ca.ibm.com
Thomas Richard William Scogland scogland1@llnl.gov
Robert Geva, robert.geva@intel.com
TBD

History

This document is a revision of N4324, adding SIMD implementation options from Pablo's earlier work and OpenMP experience from Michael Wong.

Introduction

Although thread-local storage (TLS) has a decades-long history of easily solving difficult concurrency problems, many people question its usefulness, as indeed happened at the 2014 C++ standards committee meeting at UIUC. Questioning the usefulness of TLS is especially popular among those trying to integrate SIMD or GPGPU processing into a thread-like software model. In fact, many SIMD thread-like implementations have the SIMD lanes all sharing the TLS of a single associated thread, which might come as a bit of a shock to someone expecting accesses to non-exported TLS variables to have their traditional data-race freedom. A number of GPGPU vendors are looking to use similar shared-TLS approaches, which suggests revisiting the uses of and purposes for TLS.

To that end, and as requested by SG1 at the 2014 UIUC meeting, this paper will review common TLS use cases (taken from the Linux kernel and elsewhere), look at some alternatives to TLS, enumerate the difficulties TLS presents to SIMD and GPGPU, and finally list some ways that these difficulties might be resolved.

Common TLS Use Cases

This survey includes use of per-CPU variables in the Linux kernel, as these variables are used in a manner analogous to TLS in user applications. There are more than 500 instances of statically allocated per-CPU variables, and more than 100 additional instances of dynamically allocated per-CPU variables.

Perhaps the most common use of TLS is for *statistical counting*. The general approach is to split the counter across all threads (or CPUs or whatever). To update this split counter, each thread modifies its own counter, and to read out the value requires summing up all threads' counters. In the common case of providing occasional statistical information about extremely frequent events, this approach provides extreme speedups. A number of variations on this theme may be found in the counting chapter of "Is Parallel Programming Hard, And, If So, What Can You Do About It?", including some implementations featuring fast reads in addition to fast updates. This use case typically becomes extremely important when you have more than a few tens of threads each handling streams of short-duration processing. For example, in kernels, the need for this use case often appears in networking.

Another common use of TLS is to implement *low-overhead logging or tracing*. This is often necessary to avoid creation of "heisenbugs" when doing debugging or performance tuning. Each thread has its own private log, and these per-thread logs are combined to obtain the full log. If global time order is important, some sort of timestamping is used, either based on hardware clocks or perhaps on something like Lamport clocks, though Lamport clocks are normally only used in distributed systems.

TLS is often used to implement *per-thread caches for memory allocators*, as described in this revision of the 1993 USENIX paper on memory allocation, and also as implemented in tcmalloc and jemalloc. In this use case, each thread maintains a cache of recently-freed blocks, so that subsequent allocations can pull from this cache and avoid expensive synchronization operations and cache misses. The Linux kernel uses similar per-CPU caching schemes for frequently used security/auditing information, nascent network connections, and much more.

Language runtimes often use TLS to *track exception handlers*. permitting this state to be updated and referenced efficiently, without expensive synchronization operations. TLS is also heavily used to implement `errno` and *track setjmp/longjmp state*. Some compilers also use TLS to maintain per-thread state variables. Compilers for less-capable embedded CPUs that lack a native integer-divide instruction use TLS to implement a local computational cache, especially for small-divisor cases. There are many

other examples of *tracking per-thread state*, for example, in the Linux kernel, generic sockets for packet-based communications, state controlling per-thread I/O heuristics, timekeeping, watchdog timers, energy management, lazy floating-point unit management, and much else besides.

The previous paragraph described purely local tracking of per-thread state, but it is not infrequently necessary to make that state available to other threads. Examples include quiescent-state tracking in userspace RCU, idle-thread tracking in Linux-kernel RCU, lightweight reader-writer locks ("lglocks") in the Linux kernel (but equally applicable to userspace code), control blocks for probes, such as those found in the Linux-kernel "kprobes" facility (but equally applicable to probing of userspace applications), and data guiding load-balancing activity.

Various forms of *thread ID* are typically stored in TLS. These are often used as array indexes (which is an alternative form of TLS), tie-breakers in election algorithms, or, at least in textbooks, for things like Peterson locks.

It is sometimes argued that some sort of control block should be used instead of TLS, so it is worth reviewing a synchronization primitive that provides control blocks that contain pointers to dynamically allocated per-CPU variables, namely "sleepable read-copy update," which is also known as SRCU. Each SRCU control block (AKA `struct srcu_struct`) represents an SRCU domain. SRCU readers belonging to a given domain block only those grace periods associated with that same domain. So the `struct srcu_struct` control block represents a single SRCU domain, and the dynamically allocated per-CPU state is used to track SRCU readers for that domain. This same pattern of distinct data structures containing per-CPU state occurs in quite a few other places in the Linux kernel, including networking, mass-storage I/O, timekeeping, virtualization, and performance monitoring.

In short, TLS is very heavily used, and any changes in its semantics should avoid invalidating these heavily used historic use cases.

Alternatives to TLS

A number of alternatives to TLS have been proposed, including use of the function-call stack, passing the state in via function arguments, and using an array indexed by some function of the thread ID. Although these alternatives can be useful in some cases, they are not a good substitute for TLS in the general case.

The function-call stack is an excellent (and heavily used) alternative to TLS in the case where the TLS data's lifetime can be bounded by the lifetime of the corresponding stack frame. However, this alternative does not work at all in the many cases where TLS data must persist beyond the lifetime of that stack frame.

This lifetime problem can be surmounted in some cases by allocating the TLS data in a sufficiently long-lived stack frame, then passing a pointer to this data in via additional arguments to all intervening functions. These added arguments can be problematic, particularly when the TLS data is needed by a library function. In this case, the added function arguments will often represent a gross violation of modularity.

An array indexed by some function of the thread ID can clearly provide per-thread data, however this approach has some severe disadvantages. For example, if the array is to be allocated statically, then the maximum number of threads must be known in advance, which is not always the case. Of course, a common response to uncertainty is to overprovision, which wastes memory. Software-engineering modularity concerns will often require that there be many such arrays, and performance and scalability concerns will usually require that the array elements be cacheline aligned and padded, which wastes even more memory. Finally, array indexing into multiple arrays is often significantly slower than TLS accesses.

In short, although there are some alternatives that are viable substitutes for some uses of TLS, there remain a large number of uses for which these substitutes are not feasible.

TLS Challenges to SIMD Units and GPGPUs

So why is TLS a problem for SIMD units and GPGPUs?

One problem is that large programs can have a very large amount of TLS data, and in C++ programs, many of these TLS data items will have constructors and destructors. Spending many milliseconds to initialize and run constructors on many megabytes of TLS data for a SIMD computation that takes only a few microseconds is clearly not a strategy to win—or even a strategy to break even. People working on SIMD have therefore chosen to have TLS accesses target the enclosing thread, shock and awe due to introduced data races notwithstanding. Although GPGPUs often execute in longer timeframes than do SIMD units, similar issues apply.

In addition, GPGPUs can have very large numbers of threads, which could mean that the memory footprint of fully populated per-GPGPU-thread TLS might be considered excessive in some cases.

Given the well-known problems that data races can introduce, it seems well worthwhile to spend some time looking for

alternatives, which is the job of the next section.

Can TLS and SIMD/GPGPU be Reconciled?

The old adage “If it hurts when you do that, don't do that!” suggests that SIMD units and GPGPUs should simply be prohibited from accessing TLS data, perhaps via an appeal to undefined behavior. However, given the wide range of TLS use cases, this approach seems both unsatisfactory and short-sighted. In particular, `errno` poses a particular challenge given that it is used by many library functions: Either APIs would need to change or SIMD units and GPGPUs would need to be restricted to `errno`-free portions of STL. Of course, STL implementations in which the C++ allocators use C's malloc() will make very heavy use of `errno`, in which case restricting SIMD units and GPGPUs to the `errno`-free portions of STL might be overly constraining, requiring custom allocators for every STL container, and further forbidding use of algorithms which allocate scratch memory.

In OpenMP, global data is shared by default. But in some situations we may need, or would prefer to have, private data that persists throughout the computation. This is where the `threadprivate` directive comes in handy.

The effect of the `threadprivate` directive is that the named global-lifetime objects are replicated, so that each thread has its own copy. Put simply, each thread gets a private or “local” copy of the specified global variables (and common blocks in case of Fortran). There is also a convenient mechanism for initializing this data if required. Among the various types of variables that may be specified in the `threadprivate` directive are pointer variables in C/C++ and Fortran and allocatables in Fortran. By default, the `threadprivate` copies are not allocated or defined. The programmer must take care of this task in the parallel region.

The values of `threadprivate` objects in an OpenMP program may persist across multiple parallel regions, so this data cannot be stored in the same place as other private variables. Some compilers implement this by reserving space for them right next to a thread's stack. Others put them on the heap, which is otherwise used to store dynamically allocated objects. Depending on its translation, the compiler may need to set up a data structure to hold the start address of `threadprivate` data for each thread: to access this data, a thread would then use its threadid and this structure to determine its location, incurring minor overheads. The IBM compilers implement it using the underlying TLS variable.

In order to exploit this directive, a program must adhere to a number of rules and restrictions. For it to make sense for global data to persist, and thus for data created within one parallel region to be available in the next parallel region, the regions need to be executed by the “same” threads. In the context of OpenMP, this means that the parallel regions must be executed by the same number of threads. Then, each of the threads will continue to work on one of the sets of data previously produced. If all of the conditions below hold, and if a `threadprivate` object is referenced in two consecutive (at run time) parallel regions, then threads with the same thread number in their respective regions reference the same copy of that variable. We refer to the OpenMP standard (Section 2.8.2) for more details on this directive.

- Neither parallel region is nested inside another parallel region.
- The number of threads used to execute both parallel regions is the same.
- The value of the dyn-var internal control variable is false at entry to the first parallel region and remains false until entry to the second parallel region.
- The value of the nthreads-var internal control variable is the same at entry to both parallel regions and has not been modified between these points.

The copyin clause provides a means to copy the value of the master thread's `threadprivate` variable(s) to the corresponding `threadprivate` variables of the other threads. Just as with regular private data, the initial values are undefined. The copyin clause can be used to change this situation. The copy is carried out after the team of threads is formed and prior to the start of execution of the parallel region, so that it enables a straightforward initialization of this kind of data object. The clause is supported on the parallel directive and the combined parallel work-sharing directives. The syntax is `copyin(list)`. Several restrictions apply. We refer to the standard for the details.

OpenMP 4.0 has also added support for GPGPU/accelerators and SIMD in a high-level format. In this release, we have chosen the first alternative solution outlined in this paper: prohibit interaction of `threadprivate` in GPU offloaded regions. OpenMP 4.0 states that the behaviour is unspecified:

- [80:17]: The effect of an access to a `threadprivate` variable in a target region is unspecified.
- [84:19] `threadprivate` variable cannot appear in a declare target directive.

We know a large number of OpenMP programs need `threadprivate` and this solution is merely a placeholder until we can fully explore the solution space.

In the task-oriented n3487 working paper, Pablo Halpern recommended having multiple flavors of TLS data, at the `std::thread` level, at the task level, and at the worker-thread level (this last being the context in which tasks run). It is possible that a similar approach might work for GPGPUs and SIMD units, with added keywords or other indicators as to which execution agent a given TLS data item is to be associated. This approach will require some care to avoid excessive syntactic and implementation

complexity.

Another approach would simply document the problem, for example, by providing per-lane (SIMD) and per-hardware-thread (GPGPU) TLS, but noting that provisioning large quantities of TLS, and most especially TLS with constructors, will slow things down. Unfortunately, this choice effectively rules out use of SIMD and GPGPUs by large and complex programs, which are arguably the programs most in need of the speedups often associated with SIMD and GPGPUs.

The option of simply having each lane (SIMD) or hardware thread (GPGPU) initialize the data and run the constructors suggests itself, and in some cases, this might work extremely well. However, memory bandwidth is still an issue for large programs with many megabytes of TLS (especially for short SIMD code segments). Furthermore, constructors often contain calls to memory allocators and other library functions or system calls that the hardware might not be set up to execute. The usual strategy for dealing with an operation that the hardware is not set up to execute is to delegate that operation to the enclosing `std::thread`, which simply re-introduces the bottleneck.

In cases where the constructor/destructor overhead is the main problem, it might be worth considering provisioning the TLS storage, but simply refusing to run any non-trivial constructors or destructors, perhaps issuing a diagnostic if a TLS variable with a non-trivial constructor was used.

A less radical variant of this “don't run non-trivial constructors” approach is to associate the TLS variable's lifetimes with that of the enclosing `std::thread`, so that non-trivial constructors corresponding to a given SIMD lane or GPGPU thread are run only at program start, and the variables reused by successive code fragments assigned to SIMD lanes and to GPGPU threads. More generally, if there are several levels of execution agent, it may make sense to separate the ownership and lifetime concerns, so that a given set of TLS variables is owned at a given time by an execution agent at a given level, but the lifetimes of those variables is set by a lower-level execution agent, for example, by an underlying `std::thread`.

Another option leverages the fact that code fragments handed off to SIMD lanes or GPGPU threads typically use only a tiny fraction of the TLS data. In addition, code must normally be separately generated for SIMD lanes and GPGPU threads, which might mean that the TLS offsets need not be identical to those for `std::thread`. This suggests that a code fragment handed off to SIMD or GPGPU populate only those TLS data items actually used by that code fragment. In most cases, only a small percentage of the data items will actually be used, and often that percentage will in fact be 0%.

This same strategy could be applied to `std::thread` as well. In some cases, the TLS offsets would need to be unchanged, but TLS data items at the beginning and at the end of the range could safely be omitted, and constructors could be run only on those TLS data items that were actually used.

This strategy is of course not free of issues. Here are a few of them:

1. Some of the TLS data items might be used by other threads. For example, a C++ RCU implementation might use a constructor to maintain a linked list of all tasks, which would then be used in grace-period computations. A naive analysis might leave out the TLS node used in the list because it is not referenced by `rcu_read_lock()` and `rcu_read_unlock()`, which would cause RCU to fail to observe RCU read-side critical sections running on SIMD units and on GPGPUs. Here are some possible ways of addressing this issue:
    a. As above, but use attributes or other annotations indicating dependencies from one TLS data item to any others that it might depend on.
    b. As above, but instead of using attributes or annotations, implement these dependencies via the constructors. For example, the constructor for the TLS counter used by `rcu_read_lock()` and `rcu_read_unlock()` could reference the current task's linked-list node, thus forcing it to be included in the set of TLS data items to be populated.
2. A given library might have inline functions in a header file that get compiled as SIMD or GPGPU code, and separately compiled functions that are always delegated to the associated `std::thread`. The library would reasonably expect that the same TLS data item updated by the inline function would be accessible to the corresponding separately compiled function, and might well be fatally disappointed to learn otherwise. This sort of situation could in some cases be handled by marshalling the relevant TLS data from the SIMD unit or GPGPU thread to the `std::thread` and back, although linked data structures hanging off of TLS data items would need special handling. This problem is arguably not really a problem with TLS data, but rather one of possibly many symptoms of silently switching among a group of related execution agents, which suggests that the switch not be silent. Doing things behind the developer's back is after all often a recipe for trouble.
3. There might be difficult decisions between initializing large quantities of TLS data for a given library on the one hand or delegating execution of all functions in that library to a `std::thread` on the other. Perhaps annotations could allow the developer to help (or, as the case might be, hinder) this decision process.

Another possibility, suggested by Robert Geva, is for any object (including TLS objects) defined outside of the loop to be considered common to all iterations. This means that any attempt by multiple iterations to modify an object defined outside of the loop would be considered undefined behavior.

Given that SIMD and GPGPU devices often operate in a data-parallel manner across dense arrays, another approach is to create TLS variables that are arrays, so that each SIMD lane or GPGPU thread uses the corresponding element of the TLS array. Manual

annotation seems likely to be required to support this use case. Tom Scogland notes that this approach has been used within the OpenMP community.

A final approach leverages the as-if rule. This approach takes the view that the code offloaded to SIMD units or to GPGPUs is executing as if it ran within the context of the enclosing `std::thread`, and therefore that any offloaded execution correspond to a valid `std::thread` execution. There are several cases to consider:

1. The TLS data is used only as normal non-atomic variables within the context of the enclosing `std::thread`, and all uses of a given TLS variable are ordered, for example, via dependencies. In this case, the SIMD or GPGPU code must respect these dependencies, as has in fact traditionally been the case, given the long-standing relationships between SIMD code generation and loop dependencies.
2. The TLS data is used only as normal non-atomic variables within the context of the enclosing `std::thread`, but some uses of a given TLS variable are unordered, for example, they are used in code for different actual parameters to a given function invocation. In this case, the SIMD or GPGPU code is free to reorder accesses, but if the accesses are carried out concurrently by different SIMD lanes or GPGPU threads, the compiler will need to treat the variable as if it was atomic with `memory_order_relaxed` accesses. This of course has “interesting” consequences for TLS variables that are too big to fit into a machine-sized word.
3. The TLS data is atomic, and is access and/or updated by at least one other `std::thread`. In this case, the SIMD or GPGPU code can concurrently update the TLS data, but only if the rules of atomic variables are followed—or at least if the resulting program behaves as if the rules were followed. This still has “interesting” consequences for TLS variables that are too big to fit into a machine-sized word. The default `memory_order_seq_cst` might be inconvenient in this case.

The fact that SIMD vendors were willing to expose user code to unsolicited undefined behavior might indicate that this approach is considered to be too confining.

Summary

This document has presented a number of TLS use cases, discussed some alternatives to TLS, listed some challenges faced by those combining SIMD units or GPGPUs with TLS, and looked at some possible ways of surmounting these challenges.

Acknowledgements

This proposal has benefitted from review and comment by Jens Maurer, Robert Geva, Olivier Giroux, Matthias Kretz, and Hans Boehm.